

LLD Practice Platform

Project README & AI Usage Report (Combined Submission Document)

Hiring Assignment: CipherSchools 2-Day Engineering Challenge	Date: September 2026
Candidate / Author: Rahul (akcodes-py)	Document Purpose: Single Upload for Google Form
Repository: https://github.com/akcodes-py/LLD--practice-Platform	Core Stack: React 18, TypeScript, Django 5, DRF, PostgreSQL

PART 1 — PROJECT DOCUMENTATION & RUN GUIDE

1. System Overview & The Practice Loop

A focused Low-Level Design (LLD) practice and automated evaluation platform built for the CipherSchools 2-Day Engineering Challenge. The platform enables candidates to practice interview-grade LLD problems in a continuous learning loop: **Choose Problem → Formulate Structured Design → Submit → Receive Explainable Feedback → Review History → Retry & Improve.**

5 Interview-Grade Problems Catalog:

- Parking Lot:** Multi-floor spot sizing, Strategy pattern allocation, barrier gates, fee calculation.
- Elevator System:** Dispatching algorithms (LOOK/SCAN), internal/external floor calls, motion states.
- Vending Machine:** State Pattern lifecycle (Idle, HasMoney, Dispensing, SoldOut), coin verification.
- Library Management:** Book cataloging, hold queues, borrowing limits, fine calculation rules.
- Food Ordering System:** Menus, cart lifecycle, payment gateway adapter, order delivery states.

2. System Architecture & Tech Stack

Designed as a clean, cohesive modular monolith with strict separation of domain rules from HTTP handling:

- **Frontend:** React 18 SPA + Vite + TypeScript + Tailwind CSS (responsive down to 360px).
- **Backend Monolith:** Django 5 + Django REST Framework; views are thin controllers; all validation, state transitions, and background evaluation threads reside in services.py.
- **Persistence:** SQLite (WAL mode + busy_timeout for local dev); PostgreSQL 16 for Docker Compose.
- **Async Evaluation:** Atomic DB persistence before thread launch → submit returns 202 Accepted → frontend polls /status/ every 1.5s → feedback displayed.

3. How to Run the Project (Step-by-Step)

Option A: Zero-Config Local Setup (Recommended)

```
# 1. Terminal 1 – Backend (http://127.0.0.1:8000)
cd backend
pip install -r requirements.txt
python manage.py migrate
python manage.py seed_problems # Populates the 5 problems idempotently
python manage.py runserver

# 2. Terminal 2 – Frontend (http://localhost:5173)
cd frontend
npm install
npm run dev
# Note: Vite dev server automatically proxies /api to Django. No CORS setup needed!
```

Option B: Docker Compose

```
copy .env.example .env
docker compose up --build
# Frontend live at http://localhost:8080 (SPA served via Nginx reverse proxy)
```

4. REST API Specification

Standard API error envelope: {"error": {"code", "message", "details"}}.

Method & Route	Functionality	Status Codes
GET /api/health/	System health & database connectivity check	200 OK
GET /api/problems/	List 5 practice problems with summary & difficulty	200 OK
GET /api/problems/<slug>/	Full brief (requirements, constraints, edge cases)	200, 404
POST /api/attempts/	Create draft attempt (idempotency key supported)	201, 200 (dedup)
PATCH /api/attempts/<id>/	Autosave draft sections (rejected if not draft)	200, 409 Conflict
POST /api/attempts/<id>/submit/	Persists submission & starts async evaluation	202 Accepted, 400
GET /api/attempts/<id>/status/	Polling endpoint for attempt lifecycle & results	200 OK, 404
POST /api/attempts/<id>/retry/	Clones content into new draft (history preserved)	201 Created
POST /api/attempts/<id>/re-evaluate/	Re-runs evaluator on failed evaluations	202 Accepted, 409

5. Test Suite Verification

- **Backend (Pytest):** 13 passed, 0 failures covering full API journeys, section validation checks, duplicate prevention, failure recovery, and evaluator schema contracts.
- **Frontend (Vitest & TSC):** 4 passed, 0 failures covering router smoke tests, status badges, rubric labels, and zero-error TypeScript build (tsc --noEmit && vite build).

6. Known Limitations & Production Roadmap

- **Background Threading:** Local async evaluation runs via daemon threads. For multi-tenant scale, migrate to a persistent EvaluationJob table polled by dedicated workers, or Celery + Redis.
- **Authentication:** Intentionally omitted for the 2-day MVP to prioritize the core practice loop. All attempt history is local to the instance.
- **Concurrency:** Local dev uses SQLite WAL with retry handlers; production deployments use PostgreSQL.

PART 2 — AI USAGE REPORT (AI_USAGE.md)

In accordance with Section 6 of the assignment brief, AI assistance (Claude/ChatGPT-style tools) was actively utilized as a sounding board during system design and implementation. Below is the explicit report on **5 meaningful AI-assisted decisions**, documenting what the AI suggested, what was accepted or rejected, and the engineering rationale.

Decision 1: Submission Format — Structured Text over Code / Diagram Editors

- **AI Suggested:** Support full code and diagram canvas uploads from day one to provide 'richer candidate evidence'.
- **Accepted in Part / Rejected in Full:** Chose a six-section structured text submission format only.
- **Engineering Rationale:** Code execution requires language compiler sandboxes and test runners; diagram editors add heavy canvas manipulation without increasing signal on design judgment. The assignment brief emphasizes the *smallest sufficient format*. Six structured text sections capture requirements, class responsibilities, interfaces, and trade-offs directly, which is what LLD interviews actually test.

Decision 2: Evaluation Contract — Rubric with Mandatory Quoted Evidence

- **AI Suggested:** Return an overall 0–100 score with free-text critique paragraphs.
- **Accepted with Critical Modifications:** Enforced an 8-criterion rubric where evidence (quoted directly from the submission) and confidence are strictly required fields.
- **Engineering Rationale:** Unconstrained scores drift between runs and offer vague advice. Forcing the evaluation contract to cite verbatim sentences from the learner's text guarantees actionable feedback and makes deterministic heuristics directly comparable with LLM responses.

Decision 3: Deterministic Rule-Based Default with Optional LLM (Not AI-First)

- **AI Suggested:** Call an external LLM API for every evaluation to give 'deeper qualitative critique'.
- **Rejected as Default, Accepted as Opt-In:** The default evaluator is a deterministic, rule-based heuristic engine working offline in <10ms, truthfully labelled as 'Rule-based review'. The LLM path is strictly opt-in via LLM_ENABLED=1.
- **Engineering Rationale:** Candidates and reviewers running the project locally have no API keys configured. Fabricating fake AI reviews is dishonest. The assignment rewards separating deterministic checks from judgment and gracefully handling provider failures.

Decision 4: Async Evaluation via Daemon Thread + Polling (Not a Distributed Broker)

- **AI Suggested:** Install Celery + Redis or RabbitMQ to build a 'production-ready distributed job queue'.
- **Accepted with Changes (Simplified):** Implemented async evaluation via Python daemon threads with DB persistence before thread launch, returning 202 Accepted, and frontend client polling every 1.5s.
- **Engineering Rationale:** Introducing Celery and Redis violates the assignment's explicit instruction: *'Keep this practical; do not turn the assignment into a distributed-systems project.'* The daemon thread achieves the exact same non-blocking UX with zero extra infrastructure. Database rows are committed before thread spawn, ensuring zero data loss if a thread crashes.

Decision 5: UI Restraint Over Dashboard Clichés

- **AI Suggested:** Include animated gradient hero sections, analytics stat cards, decorative charts, and community social tabs.
- **Rejected:** Built a focused, accessible light theme (CipherSchools orange #E07D18, slate typography, clear hierarchy) centered entirely on the practice and review loop.
- **Engineering Rationale:** The 40-point checklist prioritizes mobile readability (down to 360px), color contrast, and clarity over decorative noise. The interface emphasizes the problem brief, autosaving editor, score progress bars, and expandable feedback panels.

Submission Summary:

This document consolidates the complete project documentation (README) and the AI Usage Disclosure (AI_USAGE.md) into a single verified submission file. All source code, database seed scripts, unit tests, Docker configurations, and architectural notes are version-controlled at:

■ <https://github.com/akcodes-py/LLD--practice-Platform>